Распределённая обработка графов

Оценка масштабируемости D-Galois

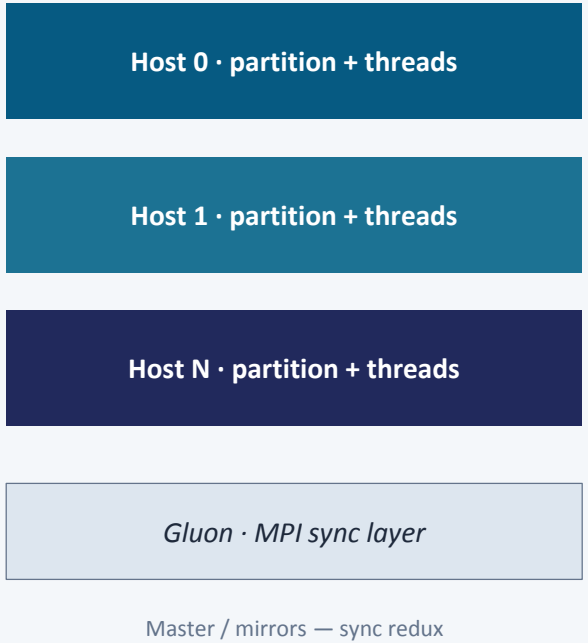
BFS · SSSP · PageRank · Triangle Counting

Ахметов Темерлан Витальевич · 2026

Что такое D-Galois

Распределённый графовый движок от UT Austin (ISS group)

- C++ библиотека параллельной обработки графов
- Распределённость через подложку Gluon (PLDI 2018)
- MPI-коммуникация между узлами кластера
- Партиционирование графа: edge-cut (oec, iec) и vertex-cut (cvc)
- Готовые приложения: BFS, SSSP, PageRank, CC, BC, k-core, TC
- Bulk Synchronous Parallel (BSP) или асинхронная связь



Вопрос исследования

?

Насколько хорошо D-Galois масштабируется при увеличении числа MPI-процессов и одинаково ли это работает для разных классов задач?

Два измерения этого вопроса

1. Насколько линейна зависимость времени от числа процессов?

2. Одинаково ли масштабируются разные классы алгоритмов?

Гипотеза

При увеличении числа MPI-процессов N время обработки графа сократится почти линейно ($S(N) \approx N$) до тех пор, пока коммуникационные накладные расходы не превысят выигрыш параллелизма.

Подгипотезы:

- Compute-bound алгоритмы (PageRank, Triangle Counting) масштабируются лучше, чем frontier-based (BFS, SSSP)
- Существует «sweet spot» N, после которого эффективность E(N) резко падает
- На single-node эмуляции ограничение масштабируемости — пропускная способность RAM, а не сеть

Графы для эксперимента

Зачем два разных графа

LiveJournal

для BFS, SSSP, PageRank

- Соцсеть LiveJournal (SNAP)
- |V| = 4 847 571
- |E| = 68 993 773 (направленные)
- Степенное распределение — типично для соцсетей
- .gr = 301 MB · .tgr = 301 MB
- Для SSSP — версия с весами uint32 [1..100]

com-Orkut

для Triangle Counting

- Сообщества Orkut (SNAP)
- |V| = 3 072 627
- |E| = 117 185 083 (неориентированные)
- Очищен от self/multi-edges (gr2cgr)
- .sgr = 918 MB
- TC требует строго симметричного графа

Источник: Stanford Network Analysis Project (SNAP)

Четыре класса задач анализа графов

BFS

Breadth-First Search

Обход графа в ширину от исходной вершины. Frontier-based: на каждой итерации обновляется уровень. Лёгкая работа, частая синхронизация.

SSSP

Single-Source Shortest Path

Кратчайшие пути от одной вершины до всех остальных. Delta-stepping. Worklist-driven, нагрузка распределена неравномерно.

PageRank

PR (pull variant)

Итеративное обновление ранга. Каждая итерация трогает все рёбра, регулярная нагрузка, плотные коммуникации.

Triangle Counting

TC (mining)

Подсчёт треугольников. Compute-bound: для каждой пары рёбер ищем общих соседей. Минимум коммуникаций между раундами.

Методология эксперимента

Что варьируем

- Число MPI-процессов N ∈ {1, 2, 4, 8}
- Алгоритм: BFS, SSSP, PageRank, TC
- = 16 конфигураций × 3 повтора = 48 замеров

Что фиксируем

- Граф (см. слайд 5)
- Потоков на процесс: t = ⌊nproc/N⌋ − 1
- Партиционирование: oec (edge-cut)
- Связь: Sync (BSP)

Метрики

T(N)
медиана времени из 3 runs (мс)

S(N)
speedup = T(1) / T(N)

E(N)
efficiency = S(N) / N

Окружение эксперимента

Single-node эмуляция кластера: каждый MPI-процесс играет роль «узла»

OS	Ubuntu 24.04 LTS (WSL2 на Windows 11)
CPU	Intel Core i5-11400H @ 2.70 GHz
Логических ядер	12 (6 физических × HyperThreading)
RAM	7.6 GiB
Galois	master, commit b67f942 (флаг GALOIS_ENABLE_DIST=1)
MPI runtime	Open MPI 4.1.6
Компилятор	gcc 13.3, CMake 3.28

Speedup и Efficiency — что это

Speedup S(N)

$$S(N) = T(1) / T(N)$$

«Во сколько раз быстрее на N процессах, чем на одном»

Efficiency E(N)

$$E(N) = S(N) / N$$

«Какая доля каждого процесса работает полезно»

Идеал vs реальность

Идеал: S(N) = N, E(N) = 1.0 — каждый процесс работает на полную

Реальность: S(N) < N из-за коммуникации, синхронизации, начала/конца параллельной фазы

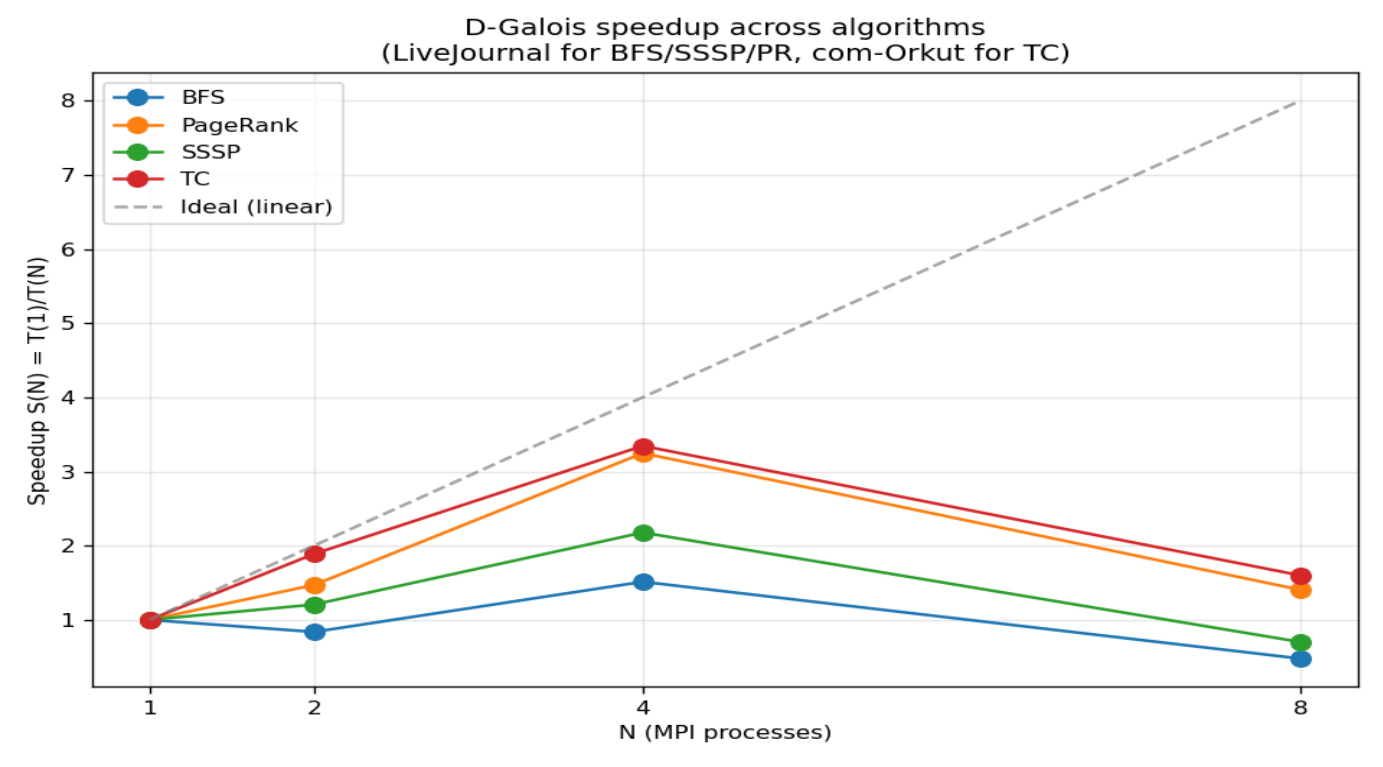
Регрессия: S(N) < 1 — параллелизм всё замедлил (typical case при oversubscribe)

Результаты и speedup

Алгоритм	T(1)	T(2)	T(4)	T(8)	max S(N)	при N
BFS	1.35 s	1.61 s	0.89 s	2.83 s	1.51×	4
SSSP	6.67 s	5.53 s	3.06 s	9.51 s	2.18×	4
PageRank	87.9 s	59.8 s	27.1 s	62.6 s	3.25×	4
TC	358.0 s	189.6 s	107.0 s	223.9 s	3.35×	4

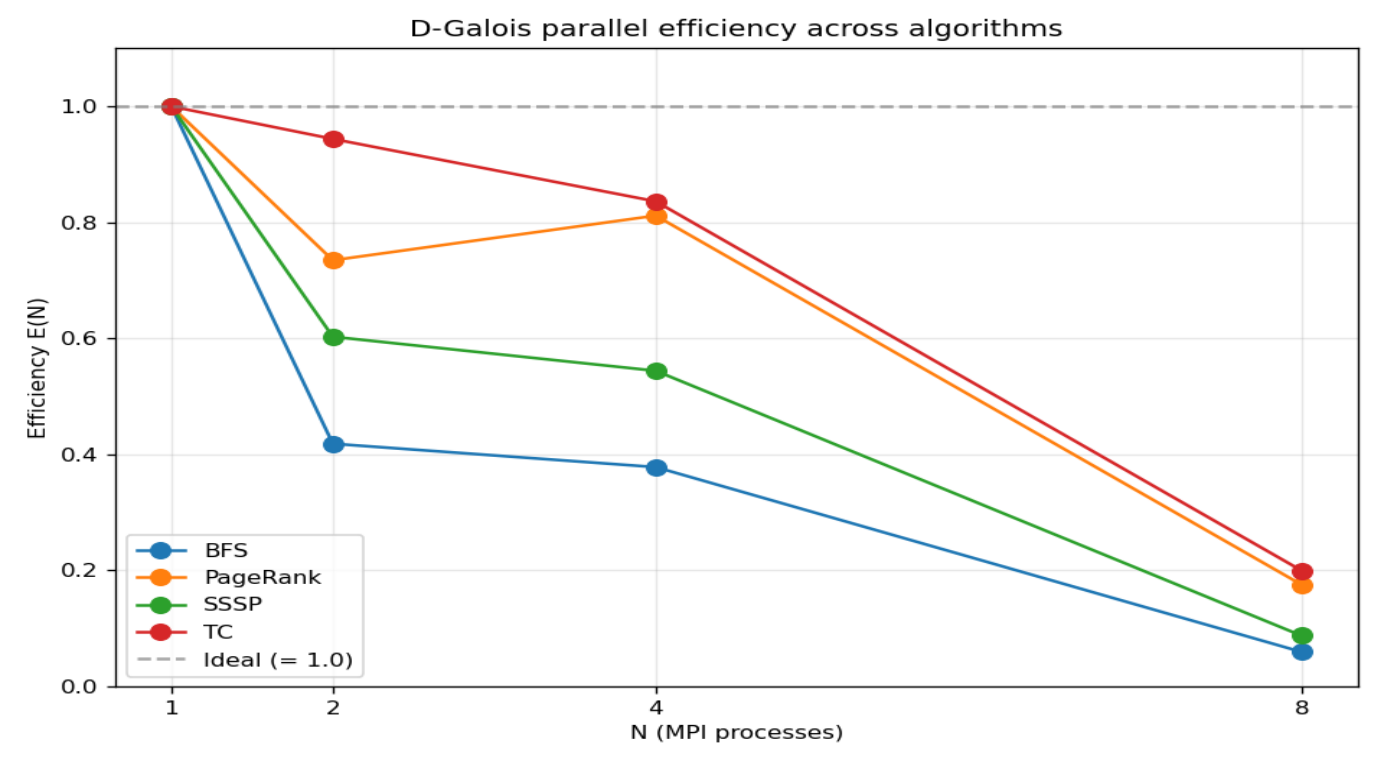
Sweet spot: у всех 4 алгоритмов лучший speedup достигается при **N = 4**. При N = 8 — деградация (oversubscribe на 12 ядрах).

Speedup всех алгоритмов



PageRank и Triangle Counting растут почти линейно до N=4 · при N=8 у всех — резкое падение

Efficiency: КПД параллелизма



TC при N=2: E = 0.94 — почти идеально · BFS на малом графе: E падает до 0.06 при N=8

Интерпретация и выводы

1

D-Galois даёт измеримый speedup

На всех 4 классах задач, max 3.35× при N=4 (Triangle Counting на com-Orkut).

2

Compute-bound > Frontier-based

TC и PR масштабируются лучше BFS и SSSP. Соотношение «полезная работа / коммуникация» — выше.

3

Sweet spot — N=4

При nproc=12 это оптимум. N=8 уже даёт oversubscribe и регрессию (1.5–2× медленнее, чем N=4).

4

Малые задачи — не для распределения

BFS на LJ за 1.3 с: накладные расходы Gluon (~3 с GraphConstructTime) делают параллелизм невыгодным.

Спасибо за внимание! · Вопросы?